Milestone 2 Summary Document

Nexus Restaurant Management Platform Rebuild — Code Audit of Sprint 2 (NestJS Backend, Multi-Tenancy, Authentication, and Core Operations)

Prepared for	Clayton Linville (Product Owner) — internal use
Audit date	August 6, 2026
Deliverable audited	Repository `sweetman_standard_clone` (clone of the_sweetman_standard), backend + frontend + infrastructure, at current HEAD
Baseline compared	Original MVP — `nexus_restaurant_manager` (FastAPI/React, ~38,500 backend LoC)
Contract reference	SOW “Nexus Restaurant Management Platform Rebuild” — Sprint 2, $2,500 + $500 PM fee; delivered Aug 3, 2026; 3-day review window
Evidence basis	Full manual code review (~4,800 backend LoC, schema, 11 migrations, frontend integration layer, Docker); 138 automated tests executed against a live PostgreSQL 16 instance (46 unit + 92 e2e, all passing); 2 targeted audit probes written and executed; staging URL verified live (HTTP 200)

Executive summary

Sprint 2 is a genuinely strong delivery. The external team rebuilt the backend on NestJS + TypeScript + Prisma + PostgreSQL exactly as contracted, and the multi-tenant core — the part of this milestone that matters most — is engineered with more care than is typical for work at this price point: a three-layer isolation design (request-scoped tenant context → a Prisma query extension that injects organization filters on every query → PostgreSQL row-level security policies for connections that bypass the application), argon2 password hashing, rotating refresh-token sessions with reuse detection, and an automated test suite of 138 tests that I ran end-to-end against a live database — every one passes, including dedicated cross-tenant isolation and RLS suites.

The audit nevertheless found real problems. One is a genuine tenant-isolation breach: **GET /api/analytics/cost-by-category returns cost data aggregated across every organization on the platform** — I confirmed this live, with each of two seeded organizations receiving the combined totals of both. It exists precisely because the one query in the codebase that touches a child table directly (`SaleItem`) is invisible to the Prisma isolation layer, and the RLS layer deliberately does not apply to the application’s own connection. Second, the new recipe-costing engine **ignores measurement units entirely** — the MVP’s unit-conversion system (global + per-ingredient conversions) was not carried over, so a recipe entered in grams against an ingredient priced per kilogram silently produces a cost that is wrong by 1,000×. Third, **the email layer is a stub**: password-recovery emails are never actually sent, in any environment, although recovery is contracted Sprint 2 scope.

Recommendation: accept Sprint 2 with revisions, and raise the revision request immediately — the contractual 3-day review window for the Aug 3 delivery closes on approximately Aug 6 (today). Findings M2-H1 (isolation leak) and M2-H3 (password recovery cannot deliver email) sit squarely inside Sprint 2’s contracted scope and are fair revision requests at no extra cost. The unit-conversion gap (M2-H2) contractually belongs to Sprint 3’s costing-engine scope — it should be converted into explicit, written Sprint 3 acceptance criteria rather than a Sprint 2 dispute. Section 6 is a concrete pre-Sprint-3 checklist.

Severity	Count	Finding IDs
HIGH	3	M2-H1 (cross-tenant analytics leak, confirmed live) · M2-H2 (costing ignores units) · M2-H3 (email delivery is a stub)
MEDIUM	5	M2-M1 (COUNT overwrites org stock, confirmed live) · M2-M2 (unvalidated cross-org reference IDs) · M2-M3 (isolation is a single layer in practice) · M2-M4 (location pricing unused in costing) · M2-M5 (default-open location access)
LOW	9	M2-L1 … M2-L9 (hardening and hygiene items, section 5.3)
INFO	6	Observations, no action required (section 5.4)

1 · Scope, method and evidence

The audited deliverable maps to the SOW as follows: Sprint 1 = Milestone 1 (front-end rebuild, previously accepted without audit), Sprint 2 = Milestone 2 (this audit), Sprint 3 = document import / costing / waste / sales / reporting, Sprint 4 = predictive ordering / Stripe / QA / deployment. Because Milestone 1 was never audited, the front end was reviewed here too, focused on its integration contract with the new backend (auth cookies, silent token refresh, the location-switching header, role gating, and the typed service layer).

Method

- Read every backend source file (74 files, ~4,800 LoC), the full Prisma schema (24 models), all 11 SQL migrations, the seed script, and both Docker compose files plus both Dockerfiles.
- Reviewed the frontend API client, auth/session handling, location switching, role gating, and the Playwright smoke suite; spot-checked screens against backend contracts.
- Stood up PostgreSQL 16 locally, applied all migrations, seeded the demo dataset, and executed the team's complete test suite: **46 unit tests and 92 end-to-end tests — 138/138 pass**.
- Wrote two additional audit probes (removed afterwards): one demonstrating the cross-tenant analytics leak over live HTTP with two logged-in organizations, one demonstrating the inventory COUNT defect. Both reproduced.
- Compared the delivered food-cost logic line-by-line against the MVP's costing engine (`cost_recalculation.py`, `unit_conversions.py`) and the MVP's auth and tenancy model.
- Verified the team's staging deployment (nexus-web.acw-test.space) responds (HTTP 200).

Limitations. The delivery arrived as a single squashed "clone" commit, so per-sprint history, code review trail, and authorship could not be inspected. The team's AWS staging environment was verified reachable but its server-side configuration (secrets handling, database hosting, backups) was not inspectable from the repository; no IaC or CI definitions are present yet (the SOW schedules those for Sprint 4). Labor-cost functionality is absent from both codebases and excluded by the SOW, so no labor-cost comparison was possible (section 3.4).

2 · Progress summary — what the team built

The backend is a single NestJS application organized as compact single-file modules (DTO + service + controller per domain), a deliberate and consistent style that keeps each domain reviewable at a glance. The data model is PostgreSQL via Prisma with 24 models covering the full operational domain: organizations, locations, users, sessions, invitations, vendors, categories, ingredients (with per-location stock and per-location pricing overlays and a price-history ledger), inventory transactions, recipes (with sections, instructions, nested components), menu items, invoices with line items, sales with line-item price/cost snapshots, production batches, and waste entries. Money and quantities are stored as fixed-precision `Decimal` columns rather than floats.

The tenancy architecture (the heart of this milestone)

- **Layer 1 — request context.** Middleware verifies the JWT access cookie and publishes {organizationId, userId, role, locationId} into Node AsyncLocalStorage for the request.
- **Layer 2 — Prisma tenant extension.** Every query against a tenant-owned model automatically receives an `organizationId` filter, every create is stamped with the context organization (a forged organizationId in a request body is overridden), and list queries are additionally narrowed to the active location. Platform administrators bypass scoping explicitly.
- **Layer 3 — PostgreSQL row-level security.** Every table carries a tenant policy — including the nine child tables that reach their tenant through a parent — enforced for a dedicated non-owner role (`nexus_app`) intended for anything that connects to the database outside the application. The application itself connects as the table owner and is exempt by design (see finding M2-M3).
- **Guards.** A global auth guard (default-deny; routes opt into @Public), a role guard over the 5-tier hierarchy (viewer < staff < manager < admin < owner, matching the MVP), a location guard that validates the client-supplied X-Location-Id header against the organization and the user's per-location assignments, and a platform-admin guard that answers 404 (not 403) to hide admin routes.

Authentication and account lifecycle

Registration creates organization + owner + default location in one transaction. Login issues a 15-minute access JWT and a 7-day refresh JWT, both httpOnly SameSite=Lax cookies (the refresh cookie path-scoped to /api/auth); refresh tokens are stored server-side as SHA-256 hashes with rotation and reuse detection (a replayed old token kills the session). Five failed logins lock the account for 15 minutes. Password reset uses hashed, single-use, 1-hour tokens and revokes all sessions; password change requires the current password. Invitations are org-scoped, role-restricted (owner is not invitable), location-assignable, expiring, revocable and resendable — accepting one creates the account and signs the user in. Team management can change roles and location assignments, deactivate members (sessions revoked immediately), and cannot touch the owner or yourself.

Sprint 2 scope versus what was delivered

Sprint 2 SOW item	Status	Notes
NestJS + TypeScript + Prisma + PostgreSQL rebuild	**Delivered**	Prisma 7 with the driver-adapter setup, migrations, generated client.
Multi-tenant data model (orgs, locations, users, roles, permissions, vendors, ingredients, inventory, recipes, menus + supporting records)	**Delivered**	Also invoices — ahead of schedule. Recipe sections/instructions exist in schema only (no API yet; Sprint 3).
Registration, login, password recovery	**Partial**	Flows implemented and tested, but the mailer is a stub — recovery email can never be delivered (M2-H3).
Organization onboarding	**Delivered**	Org + owner + default location, 14-day trial.
User invitations, role assignment, revoke, permissions	**Partial**	Complete and well-tested over HTTP; email delivery stubbed (UI surfaces the accept link as a fallback).
Restaurant-location management and access	**Delivered**	Create/edit/archive/restore, per-user assignment, header-based switching validated server-side.
Vendor / ingredient / inventory / menu / recipe modules	**Delivered**	Full CRUD with org-scoped uniqueness, atomic inventory ledger, per-location stock and pricing overlays.

Sprint 2 SOW item	Status	Notes
Docker, AWS, PostgreSQL, S3, Redis foundation	**Partial**	Docker + Postgres solid (multi-stage images, healthchecks, migrate-on-start). S3 service written but unused until Sprint 3; Redis provisioned but idle until Sprint 3; no IaC/CI in repo (SOW: Sprint 4); staging is live.
Structured logging, env-based secrets, BugSnag	**Delivered**	pino with header redaction and request IDs; zod-validated environment (32-char minimum JWT secrets); BugSnag wired.
Swagger documentation + structured error handling	**Delivered**	Swagger on /api/docs; a global filter maps Prisma failures to clean 409/400/404 and reports unknowns to BugSnag.
Delivered early from Sprint 3 scope	**Bonus**	A first costing engine (see M2-H2), waste tracking, manual sales with cost snapshots, invoices CRUD, dashboard KPIs/analytics, production log.

Quality signals worth naming: the SQL migrations carry unusually candid engineering commentary (the RLS design rationale is documented in the migration files themselves); the seed builds two realistic organizations with deliberately overlapping names so isolation is testable; and the 138-test suite covers auth flows (lockout, rotation reuse, multi-device sessions), cross-tenant reads/writes/updates at both the Prisma and HTTP layers, location boundaries, costing cascades including cycle rejection, and RLS behavior under the non-owner role. This is materially better test discipline than the MVP (one manual multi-tenant script plus a small e2e folder).

3 · MVP vs. Milestone 2 comparison

Raw size is not comparable at face value: the MVP's ~38,500 backend lines include AI document extraction, infographic generation, monitoring and admin tooling that the SOW schedules for Sprints 3–4. The comparison below is limited to the ground both systems cover.

Area	MVP (nexus_restaurant_manager)	M2 (sweetman_standard_clone)
Backend stack	FastAPI + SQLAlchemy (Python), ~86 modules	NestJS + Prisma (TypeScript), 74 files, typed end-to-end
Database	SQLite in production (PostgreSQL aspirational)	PostgreSQL 17, Prisma migrations, Decimal money columns
Tenancy enforcement	Manual: every route must remember organization_id filters (documented as its #1 gotcha)	Centralized: automatic query-level injection + stamping, plus DB-level RLS for non-app connections; org-scoped uniqueness kept (e.g. name unique per org)
Sessions & tokens	30-day JWT in localStorage; bcrypt-12; no lockout, no revocation, no sessions	15-min access + 7-day rotating refresh in httpOnly cookies; argon2; lockout; server-side sessions; reuse detection; sessions revoked on password change/reset/deactivation
Locations / properties	Bolted on: accessible_properties JSON on users; property pricing table	First-class: Location entity, per-user assignments, per-location stock and pricing overlays, location-scoped records, guard-validated switching
Recipe costing	Topological cascade with unit conversions (global + per-ingredient, reverse fallback) and per-property pricing; missing conversion → cost 0 or unconverted quantity with a log warning	Topological cascade with explicit cycle rejection and a nesting-depth cap, unit-tested — but no unit conversion at all and location pricing unused (M2-H2, M2-M4)
Food-cost %	Compared against a stored target	Measured against actual menu prices, with a per-recipe target indicator
Sales	Daily totals; external_id/source columns anticipating POS	Line-item sales with point-in-time price and cost snapshots (a better basis for variance reporting); POS fields dropped (out of scope by SOW)
Inventory	Transaction log with waste reasons	Same concept, atomic multi-write transactions, org + per-location levels, price-history ledger fed by receiving and manual price changes
Documents / AI import	Working Gemini/Claude extraction pipeline in production use	Not yet (Sprint 3); S3 storage service and invoice data model already in place for it
Testing	Manual multi-tenant script + small e2e folder	138 automated tests (unit + HTTP e2e + RLS), all passing in this audit
Deployment	Lightsail, rsync + systemd scripts, SQLite file + cron backups	Docker images (multi-stage, healthchecks, migrate-on-start), compose for dev and deploy; staging live; CI/IaC pending (Sprint 4)

3.1 · Food-cost calculation — the detailed picture

Both engines share the same core idea: build the dependency graph of nested recipes, sort it topologically, and roll costs bottom-up so a price change cascades to every parent recipe. The M2 implementation is the better piece of engineering — it rejects cycles explicitly with a clear 400 ("adding this component would create a circular relationship"), caps nesting depth at 5, recalculates the whole organization in one pass, and is covered by both unit tests and HTTP e2e tests (including "an ingredient price change cascades up to the parent recipe"). The MVP relies

on the topological order alone and logs rather than prevents pathological graphs.

But the MVP is the more *correct* engine where it counts for a chef. It converts recipe-line units into the ingredient's priced unit before multiplying (gallon→cup, kg→lb, plus per-ingredient overrides such as "to taste"→teaspoon), and it prices ingredients per property when a property price exists. M2 does neither: the recipe line's unit field is stored but never read during costing — cost is simply quantity × the ingredient's organization-level cost-per-unit — and nested-component cost only scales by yield when the units happen to match as exact strings, silently falling back to per-portion math otherwise. There is no unit-conversion table in the M2 schema at all. The practical consequence: a recipe entered as "200 g" of an ingredient priced per kg costs out at 200× reality, with no warning anywhere. For a product whose pitch is accurate food-cost tracking, this is the single most important functional gap to close (finding M2-H2; the SOW places unit conversions inside Sprint 3's costing-engine scope, so this is an acceptance-criteria item for the next milestone rather than a Sprint 2 breach).

3.2 · Data handling

M2 is a clear upgrade: PostgreSQL with real migrations instead of a production SQLite file with hand-run idempotent SQL; fixed-precision Decimal columns for money instead of floats; org-scoped unique constraints preserved; a price-history ledger and per-location stock/pricing that the MVP approximated with a single property-pricing table; and an inventory ledger whose writes (entry + org stock + location stock + optional price history) are applied in a single database transaction. Two data-handling caveats: costing arithmetic itself converts Decimals to JS floats (rounded to 4 dp — acceptable at pilot scale, worth a documented policy), and per the SOW, historical MVP data is explicitly not migrated (you confirmed no migration is needed).

3.3 · Security posture versus the MVP

Every dimension improved: password hashing (bcrypt→argon2id), token lifetime (30 days→15 minutes), token storage (localStorage, XSS-stealable → httpOnly cookies), session revocation (none→server-side with rotation and reuse detection), brute-force defense (none→lockout), password reset (n/a→hashed single-use tokens), tenancy enforcement (per-route discipline→centralized injection + RLS), secrets (ad-hoc .env→schema-validated environment), error reporting (log file→BugSnag), and log hygiene (token-bearing headers redacted). No dimension regressed.

3.4 · Labor cost

For completeness against your original brief: labor-cost tracking exists in **neither** codebase. A search of the MVP backend finds no labor module (consistent with it having been theorized but never built), and the SOW expressly lists payroll, employee scheduling and advanced labor tracking as out of scope for this rebuild. If labor costing matters to the product roadmap, it will be a post-Sprint-4 change request; nothing in the M2 schema blocks adding it later.

4 · Improvements successfully implemented

- **Tenancy by construction, not by discipline.** The MVP's top documented gotcha ("always remember the organization_id filter") is now impossible to forget for every model the extension covers: filters are injected, creates are stamped, and a forged organizationId in a request body is overridden — proven by tests. Foreign rows 404 rather than 403, so record existence doesn't leak.
- **Defense-in-depth at the database.** Full RLS policy coverage — including child tables scoped through their parents and the organization root itself — enforced for a dedicated non-owner role, with the reasoning written into the migrations. Any future analytics tool, psql session or misconfigured script hitting the database directly gets tenant isolation for free.
- **Authentication of production quality.** argon2id, short-lived access tokens, rotating hashed refresh sessions with reuse detection, lockout, hashed reset tokens, revocation on password events and deactivation, no user enumeration on login or forgot-password.
- **An honest, well-tested costing core.** Cycle rejection with a friendly error, a nesting depth cap, organization-wide cascade recalculation on every relevant mutation, and food-cost percent measured against real menu prices (unit semantics still to come — M2-H2).
- **Atomic inventory.** Ledger entry, organization stock, per-location stock and price history move in one transaction; waste entries write the entry, the WASTE ledger row and both stock levels together and are valued at current cost.
- **Operational hygiene.** Structured logs with redaction and request IDs, BugSnag with tenant metadata on unhandled errors, zod-validated environment that refuses to boot on weak secrets, clean global error mapping (Prisma unique/FK/not-found → 409/400/404), health endpoint, Swagger.
- **Infrastructure that matches the SOW.** Multi-stage Docker images, container healthchecks, migrations applied on start with advisory-lock safety for parallel replicas, dev and deploy compose files, S3 service (path-traversal-safe keys, presigned GETs) ready for Sprint 3.
- **A test suite that earns trust.** 138 tests spanning unit, HTTP e2e and RLS layers — including deliberately overlapping tenant data — all green in this audit's independent run.

5 · Risks and issues

IDs follow the HIGH/MED/LOW convention from the earlier remediation trail in the MVP repository. "Confirmed live" means I reproduced the behavior against a running server during this audit.

5.1 · High

HIGH **M2-H1 — Cross-tenant data leak in analytics — confirmed live**

Where: GET /api/analytics/cost-by-category · backend/src/modules/dashboard/dashboard.module.ts:184

The endpoint aggregates `SaleItem` rows with `saleItem.findMany()` and no tenant filter. `SaleItem` is a child table (no organizationId column) and is not in the tenant extension's model list, and the RLS layer exempts the application's own connection — so nothing scopes the query. My probe logged in as each of the two seeded organizations and both received the **combined** cost-by-category totals of the entire platform, mixing both organizations' category names and sale costs.

Severity is high because it is a real tenant-isolation breach in the exact area this milestone sells ("fully protected by multi-tenant access guards" per the status report), even though the leaked data is aggregated rather than row-level. It is also the proof-case for M2-M3: one missed model in a single app-side layer becomes a leak.

Recommended fix: Scope the query through its parent (`where: { sale: { organizationId } }` or query Sale and include items), and add a regression e2e that calls every analytics endpoint as two different organizations and asserts disjoint results. In scope for Sprint 2 revision.

HIGH

M2-H2 — Recipe costing ignores measurement units; no unit-conversion system

Where: backend/src/costing/costing.core.ts · Prisma schema (no conversion model) · recipe/menu/sale cost paths

A recipe line's unit is stored but never read during costing: cost = quantity × the ingredient's org-level costPerUnit, whatever the units. Nested components scale by yield only when the edge's unit string exactly equals the component's yield unit, silently falling back to per-portion math otherwise. The MVP's conversion table (seeded volume/weight conversions, per-ingredient overrides, reverse lookups) was not ported, and no equivalent model exists in the M2 schema. A "200 g" line against a per-kg price costs out 200× too high with no warning; these numbers flow onward into menu-item costs, sale cost snapshots, dashboards and reports.

Contractually, unit conversions are listed inside the Sprint 3 costing-engine scope (SOW §15), and the Sprint 2 costing engine is early-delivered bonus work — so this is not a Sprint 2 breach. But it is the single most product-critical gap in the codebase today, and the sale/menu snapshots built on top of it will bake wrong numbers into history.

Recommended fix: Make it a written Sprint 3 acceptance criterion: a conversion model (global + per-ingredient, org-scoped), conversion applied in ingredient and component contributions, and a visible "unconvertible unit" flag instead of a silent guess — with test cases (g/kg, oz/lb, mismatch flagging). Interim, low-cost Sprint 2 revision: flag recipe lines whose unit ≠ the ingredient's unit in the API response so the UI can warn.

HIGH

M2-H3 — Email delivery is a stub — password recovery cannot work

Where: backend/src/mailer/mailer.service.ts

MailerService never sends anything: without MAIL_PROVIDER_URL it logs and returns false, and *with* it, it still only logs and returns true — there is no provider integration behind the flag. Invitations degrade gracefully (the API returns the accept link for manual sharing), but forgot-password has no such fallback: the user is told to check email that will never arrive. Password recovery and a transactional email provider are named Sprint 2 / stack-level scope in the SOW.

Secondary issue: the "configured" code path logs the full message body — including reset and invitation links — so once a real provider is wired this way, secrets would flow into logs (see M2-L9).

Recommended fix: Integrate a real transactional provider (SES, Postmark, Resend — client's account per SOW terms), keep the dev fallback, stop logging secret URLs outside development, and add an e2e that asserts provider dispatch. Fair Sprint 2 revision request.

5.2 · Medium

MEDIUM

M2-M1 — A physical COUNT at one location overwrites organization-wide stock — confirmed live

Where: POST /api/inventory/transactions (type=COUNT) · inventory.module.ts

COUNT sets the per-location stock to the counted value *and* sets the organization-level onHand to that same value. With stock at several locations, counting 2 units at one location rewrote the org total to exactly 2 in my probe. The e2e test named "a physical count keeps the location and the organization total in step" only asserts the location figure and then hard-resets the org total in cleanup, so the suite masks the defect.

Recommended fix: On COUNT, adjust the org onHand by the delta at that location (or recompute as the sum of location stocks), and make the test assert the org total.

MEDIUM

M2-M2 — Cross-organization reference IDs accepted without validation

Where: recipes.categoryId · menuItems.categoryId · invoices.vendorId · invoice items.ingredientId

The ingredients module validates vendor/category references through the tenant-scoped client, but recipes and menu items accept categoryId — and invoices accept vendorId and line-item ingredientId — straight into the write. Foreign-key checks run as the table owner and bypass RLS, and the extension cannot see nested reference fields, so a foreign organization's cuid is accepted and its name subsequently rendered through relation includes. Practical exploitability is low (cuids are unguessable), but it is an isolation-hygiene hole and internally inconsistent with the team's own pattern.

Recommended fix: One shared assertRefs helper used by every module that accepts reference IDs; an e2e asserting foreign reference IDs are rejected with 400.

MEDIUM

M2-M3 — In practice, isolation rests on a single application-side layer with no coverage guard

Where: tenancy/tenant.extension.ts (TENANT_MODELS) · RLS design (app connects as table owner, GUC never set)

The three-layer story is honest but asymmetric: for the application's own traffic, RLS is deliberately inert (owner connection, and the per-request `app.current_org` setting is never issued anywhere in the app), so the Prisma extension's hand-maintained TENANT_MODELS list is the only thing standing between a query and every tenant's data. M2-H1 is what one missed entry looks like. Nothing currently fails the build when a new model (or a direct child-table query) slips past the list.

Recommended fix: Add an automated guard: a unit test that introspects the Prisma DMMF and fails if any model with an organizationId field is missing from TENANT_MODELS, plus a lint/review rule (or wrapper) for child-model queries requiring parent-scoped where clauses. Longer-term option: run the app under the non-owner role with a transaction-scoped GUC for genuine defense-in-depth, accepting the round-trip cost.

MEDIUM

M2-M4 — Per-location ingredient pricing exists but costing never uses it

Where: costing.service.ts (reads ingredient.costPerUnit only) vs ingredients.module.ts (location pricing overlay)

The UI shows a location-specific cost when a location price exists, and receiving records per-location price history — but recipe costs, menu costs and sale cost snapshots are always computed from the organization-level price. The MVP's engine consulted property pricing during costing. Result: two locations with different supplier prices see identical "recipe cost", and the display price can contradict the costed price.

Recommended fix: Product decision, then implementation: either costing consumes location pricing (per-location recipe costs — larger change, natural to fold into the Sprint 3 costing criteria) or the UI stops implying that it does.

MEDIUM

M2-M5 — Users with no location assignment default to every location

Where: common/guards/location.guard.ts · auth /me locations fallback

Zero UserLocation rows means unrestricted: a staff member nobody got around to assigning sees all locations' data. Defensible for owner/admin; for staff/viewer the SOW language ("receives limited access based on the permissions assigned") suggests default-closed is the intent.

Recommended fix: Make non-admin roles default-closed (no assignments → no locations) or force assignment at invitation time; one-line policy change plus a test either way.

5.3 · Low

ID	Finding and recommendation
M2-L1	GET /api/auth/refresh mutates state (rotates the token pair). Safe-method semantics say POST; SameSite=Lax sends cookies on top-level GET navigations, so a link can force a rotation (nuisance, not a takeover).

ID	Finding and recommendation
M2-L2	No per-IP throttling anywhere: login/lockout is per-account only, and forgot-password, invitation preview/accept (token guessing) and registration are unthrottled. Add @nestjs/throttler before public pilot.
M2-L3	Invitation tokens are stored in plaintext (reset tokens are hashed). A database read yields usable account-creation links. Hash at rest like reset tokens.
M2-L4	Swagger UI and the full API schema are public at /api/docs in every environment. Gate behind auth or an environment flag in production.
M2-L5	Production batches record recipe cost ignoring the quantity produced (3 batches cost the same as 1) and do not deduct ingredient stock (the latter is explicitly labeled as future work in the API docs).
M2-L6	Costing math runs in JS floats over Decimal columns (4-dp rounding); receiving-derived price history rounds unit cost to 2 dp in a 4-dp column; MenuItem.cost column exists but is never written (cost is computed on read).
M2-L7	Most list endpoints are unpaginated (the pagination DTO exists and is used once). Fine at pilot volume; will matter once AI ingestion multiplies invoice/document rows in Sprint 3.
M2-L8	No security-header middleware (helmet) and no CSRF token; current posture is acceptable because of SameSite=Lax + strict CORS + cookie path scoping, but harden before broader exposure.
M2-L9	The mailer logs full message bodies including secret reset/invite URLs (see M2-H3); token-bearing links must never reach production logs.

5.4 · Informational (no action required)

- The delivery is a single squashed commit; per-sprint history and review trail are not inspectable. Consider requiring incremental pushes (or PRs) from Sprint 3 onward — it also makes future audits cheaper.
- REDIS_URL is required at boot but Redis is entirely unused until Sprint 3 (the compose file says as much).
- The dashboard’s organization-wide food-cost target is hardcoded at 30%; per-recipe targets exist and work.
- Recipe sections and instructions are modeled in the schema but have no API yet — consistent with Sprint 3’s import scope.
- Billing plans are static placeholders and the admin portal is intentionally thin (organizations list + empty queue) — both are Sprint 4 scope and honestly labeled as such in the API docs.
- Storage keys currently have no per-organization namespace convention; harmless while unused, but define one before Sprint 3 starts writing tenant documents to S3 (see checklist item 7).

6 · Pre-Milestone-3 checklist

Ordered by leverage. Items 1–6 should be complete before Sprint 3 code lands on top of them; 1, 3 and 4 are fair Sprint 2 revision requests, and 2, 5 and 6 are small enough to bundle with them. Items 7–14 should be agreed now and executed early in Sprint 3.

#	Must complete before Sprint 3 work begins	Closes
1	Fix M2-H1 and add a two-organization regression e2e over every analytics endpoint.	M2-H1 · Sprint 2 revision
2	Add the tenant-coverage guard test (every model with organizationId must appear in TENANT_MODELS; child-table queries require parent scoping) so the next missed model fails CI, not production.	M2-M3
3	Wire a real transactional email provider; keep the dev fallback; strip secret URLs from logs; e2e-assert dispatch for reset and invitations.	M2-H3, M2-L9 · Sprint 2 revision
4	Fix COUNT semantics (org total = reconciliation of location stocks) and make the existing e2e actually assert the organization total.	M2-M1 · Sprint 2 revision
5	Write unit-conversion acceptance criteria into Sprint 3 sign-off: conversion model (global + per-ingredient, org-scoped), applied in ingredient and component costing, visible “unconvertible” flags, and named test cases (g↔kg, oz↔lb, mismatch warning). Interim: flag unit mismatches in recipe APIs.	M2-H2
6	Centralize reference-ID validation (one assertRefs helper for categoryId/vendorId/ingredientId/recipeId everywhere) with a foreign-ID-rejection e2e.	M2-M2

#	Strongly recommended before/alongside early Sprint 3	Closes
7	Define the S3 tenant namespace now (keys prefixed org/<organizationId>/… enforced in StorageService) plus upload authorization rules — before Sprint 3 writes the first tenant document.	Sprint 3 readiness
8	Per-IP throttling on public endpoints (login, register, forgot-password, invitation preview/accept).	M2-L2
9	Decide the location-pricing policy: costing consumes location prices (fold into Sprint 3 costing criteria) or the UI stops implying it.	M2-M4
10	Decide default-open vs default-closed location access for unassigned staff/viewer roles.	M2-M5
11	Pagination for transactions, invoices and the future documents queue before AI ingestion multiplies rows.	M2-L7
12	Move organization-wide cost recalculation off the request path (BullMQ arrives in Sprint 3 anyway) — invoice confirmation will otherwise trigger synchronous org-wide recalcs on every upload.	Sprint 3 readiness
13	Auth hardening batch: refresh via POST, helmet, gate Swagger in production, hash invitation tokens.	M2-L1/L3/L4/L8
14	Stand up CI now (the suite is fast — the full 138 tests ran in ~16 s here) rather than waiting for Sprint 4; require incremental commits/PRs from the team for reviewability.	Process

7 · Acceptance recommendation

Accept Sprint 2 with revisions — and communicate the revision list today. Under the SOW’s terms, each deliverable auto-completes three days after delivery absent feedback; Sprint 2 was delivered August 3, so the window closes on or about August 6. The revision request should cover M2-H1, M2-H3 and M2-M1 (all inside Sprint

2's advertised scope: "fully protected by multi-tenant access guards" and "password recovery"), with items 2, 5 and 6 of the checklist attached as small bundled fixes. Treat M2-H2 and M2-M4 as written acceptance criteria for Sprint 3's costing engine rather than Sprint 2 defects — that framing is contractually accurate and keeps the relationship constructive. On the evidence of this codebase — architecture, candor of the engineering commentary, and test discipline — the team has earned the benefit of the doubt: these are fixable gaps in an otherwise well-built foundation, not signs of systemic carelessness.

Appendix A · Test-run evidence (executed during this audit)

Suite	Detail	Result
Unit — costing core	costing/costing.core.spec.ts	pass
Unit — calendar utils	common/utils/calendar.spec.ts	pass
Unit — storage keys	storage/storage.service.spec.ts	pass
Unit — tenant isolation (Prisma layer)	tenancy/tenant-isolation.spec.ts	pass
E2E — auth, sessions, invitations (HTTP)	test/auth.e2e-spec.ts	pass
E2E — modules, tenancy, locations, costing (HTTP)	test/modules.e2e-spec.ts	pass
E2E — row-level security under non-owner role	test/rls.e2e-spec.ts	pass
Total: 46 unit + 92 e2e	PostgreSQL 16, migrations + seed applied	**138 / 138 pass**
Audit probe 1 — analytics cross-tenant leak	both orgs received combined totals	**leak confirmed**
Audit probe 2 — COUNT stock overwrite	org onHand overwritten by one location's count	**bug confirmed**

Audit probes were temporary files, run locally and deleted; nothing from this audit modifies the delivered code. This report was produced with AI assistance (Claude Code) under the direction of the product owner.